Development of an Edge Computing-Based Voice and Gesture-Controlled Lighting System

Anh Q. Vu*, Hung D. Truong*, Hoang H.N. Le*, Le The Dung†
*FPT University, Ho Chi Minh City, Vietnam
†HCMC University of Technology and Engineering, Ho Chi Minh City, Vietnam
†Corresponding author: Le The Dung, Email: dunglt@hcmute.edu.vn

***Abstract*—This paper presents the design and implementation of a contactless smart lighting system controlled by both voice commands and hand gestures. The proposed architecture integrates an ESP32 microcontroller, a microphone, an optical gesture sensor, a relay-based power stage for a 12V–10W LED load, and a web-based monitoring interface. The primary contribution of this work is the deployment of an on-device keyword-spotting model based on a quantized Int8 1D convolutional neural network (1D-CNN) that uses Mel-frequency cepstral coefficients (MFCCs). To further enhance hardware-level efficiency, the inference engine leverages the Edge Impulse EON™ Compiler, which replaces generic TensorFlow Lite for microcontrollers interpreters with highly optimized, hardware-aware C++ code. By using static memory allocation and specialized compute kernels (akin to CMSIS-NN and ESP-NN), this approach bypasses traditional interpreter-based execution and significantly reduces the cycles-per-inference (CPI). Experimental results demonstrate that the proposed system achieves an overall classification accuracy of 88.2% with an ultra-low inference latency of 7 ms. Notably, this represents an approximate 38-fold reduction in processing time compared to contemporary state-of-the-art TinyML implementations on the same hardware, which typically exhibit latencies exceeding 260 ms. Furthermore, the proposed model minimizes computational overhead to a peak RAM usage of just 3.8 KB and Flash usage of 31.2 KB—achieving a highly competitive level of memory efficiency for multimodal edge interaction. The system maintains stable dynamic memory behavior during continuous operation, confirming the feasibility of deploying real-time embedded AI, multimodal interaction, and IoT communication on the ESP32's dual-core architecture.**

***Index Terms*—Edge Computing, ESP32, 1D-CNN, MFCC, Keyword Spotting, Gesture Recognition, Smart Lighting, WebSocket, IoT.**

I. Introduction

Human-machine interaction (HMI) in smart home environments is increasingly shifting from manual, mechanical interfaces to non-contact and intelligent interaction paradigms. Conventional lighting systems heavily rely on physical switches, which not only limit user convenience but also pose epidemiological risks through potential cross-contamination via shared contact surfaces. Consequently, contactless systems have gained significant traction, particularly for implementation in hygiene-critical settings such as hospitals, laboratories, and highly trafficked public spaces.

Historically, conventional smart lighting and voice-controlled assistants have relied heavily on cloud computing architectures. In such configurations, raw audio signals are continuously streamed to a central server for processing. Although effective, this approach suffers from substantial transmission latency, strict dependence on continuous network bandwidth, and severe privacy vulnerabilities. To circumvent these limitations, the paradigm of Tiny Machine Learning (TinyML) and Edge Artificial Intelligence (Edge AI) has emerged. As highlighted by Abadade *et al.* [1], integrating TinyML into Internet of Things (IoT) ecosystems enables migrating machine learning inference directly to resource-constrained edge devices. In the context of smart homes, Chittepu *et al.* [2] emphasized that TinyML embedding in voice assistants not only solves data privacy issues by localizing processing, but also significantly enhances user-centric experiences through offline real-time responsiveness.

Recent literature demonstrates the growing efficacy of deploying deep learning models on microcontrollers (MCUs) for acoustic processing. For example, Tsoo *et al.* [3] successfully deployed a speaker identification system on an ESP32 microcontroller using TensorFlow Lite, illustrating the feasibility of utilizing Convolutional Neural Networks (CNNs) and Multilayer Perceptrons (MLPs) under strict memory constraints. Similarly, González *et al.* [4] deployed a real-time speech recognition system on the ESP32, leveraging Mel-Frequency Cepstral Coefficients (MFCCs) and model quantization, achieving substantial reductions in the footprints of random access memory (RAM) and Read-Only Memory (ROM). However, deploying acoustic Edge AI requires a meticulous balance. As Kazeneza *et al.* [5] noted in their study on acoustic anomaly detection in agricultural environments, optimizing the trade-off between model complexity and predictive performance remains a critical challenge for low-power edge deployment.

Despite these advancements, existing commercial and academic solutions primarily focus on unimodal systems—either continuously processing audio [6] or exploring basic gesture-based control using infrared optical sensors [7]. Multimodal systems that successfully fuse both continuous acoustic sampling and optical gesture tracking entirely offline on a single low-power, resource-constrained Wi-Fi System-on-Chip (SoC) remain scarce. A primary technical bottleneck in previous implementations is the concurrent management of direct memory access (DMA) buffers for audio, real-time neural network inference, and stable heap memory allocation without causing real-time operating system (RTOS) fragmentation or watchdog resets.

To overcome the limitations of the prior works, this paper proposes a comprehensive embedded architecture that tightly integrates multimodal HMI and Edge AI. The proposed system significantly improves upon existing solutions, and an extensive comparison against the solution from [4] highlights the following key contributions:

1) Offline multimodal: Unlike contemporary unimodal voice-only frameworks [4], the proposed architecture seamlessly fuses I2S-based acoustic sampling with I2C-based interrupt-driven optical gesture tracking. This dual-modality approach establishes redundant interaction pathways, preserving complete offline data privacy.
2) Streamlined neural network architecture: Rather than relying on deeply layered neural networks that inflate memory requirements [4], we deploy a highly streamlined, two-layer 1D-CNN pipeline using MFCCs [8]. Using Int8 post-training quantization, the model achieves competitive classification accuracy (88.20%) while restricting its static RAM footprint to a microscopic 3.8 KB, significantly outperforming the memory efficiency of deeper contemporary architectures.
3) Bidirectional high-cadence telemetry: By embedding an event-driven WebSocket server directly into the microcontroller's flash memory [9], the system guarantees instantaneous full-duplex status synchronization. This architecture successfully bypasses the processing overhead and temporal lag inherent in traditional HTTP request-response polling, ensuring seamless IoT integration.
4) Fourfold inference acceleration: By substituting traditional TensorFlow Lite for Microcontrollers (TFLM) interpreters with the Edge Impulse EON™ Compiler, the inference engine executes directly via optimized C++ compute kernels. This approach reduces core inference latency to just 7 ms—a fourfold acceleration compared to the 28 ms inference phase reported in state-of-the-art baselines [4]—thereby freeing critical CPU cycles for concurrent multimodal tasks.

The remainder of this paper is organized as follows. Section II describes the overall system architecture, hardware integration, power management, and the IoT communication platform. Section III details the multimodal processing pipeline, encompassing both acoustic embedded AI and gesture control logic. Experimental results are analyzed in Section IV, and Section V concludes the paper.

II. System Architecture and Hardware Integration

The proposed system consists of four major subsystems: sensing, processing, communication, and actuation. The overall objective is to convert multimodal user inputs into lighting control actions with low latency and high reliability.

A. Hardware Overview

The ESP32 is selected as the main controller because it integrates a dual-core processor, wireless connectivity, and sufficient memory resources for real-time embedded AI inference. The sensing layer includes:

- INMP441 microphone captures digital audio through the synchronous I2S interface, ensuring an extremely low noise floor suitable for speech processing [10].
- PAJ7620U2 gesture sensor detects complex hand gestures using infrared optical sensing and communicates over the I2C bus [7].

The actuation layer includes an isolated relay module for switching a 12V–10W LED load and a WS2812B RGB LED strip for visual status indication [11]. Power management is implemented by an LM2596 buck converter [12] that steps down the primary 12V supply to a stable 5V for logic and auxiliary components.

B. Functional Workflow

The multimodal processing flow can be abstracted using the following control function:

$$u(t) = f\big(x_{\text{voice}}(t), x_{\text{gesture}}(t), x_{\text{web}}(t)\big), \tag{1}$$

where $x_{\text{voice}}(t)$ is the voice input stream, $x_{\text{gesture}}(t)$ is the decoded gesture command, $x_{\text{web}}(t)$ is the command received from the browser interface, and $u(t)$ is the final multiplexed control signal applied to the relay and RGB LEDs.

C. Block Diagram

Fig. 1 illustrates the system-level architecture, showing the interaction between the edge node, sensors, and the local web interface.

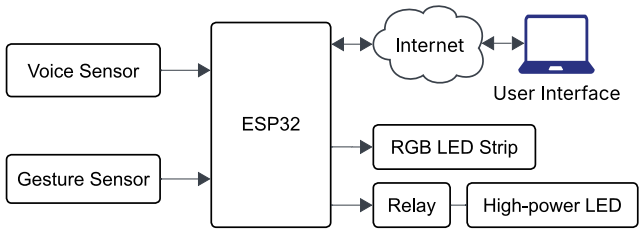


Fig. 1. Overall architecture and data workflow of the proposed multimodal system.

D. IoT Communication and Telemetry

The remote monitoring interface is deployed via an embedded web server leveraging the WebSocket protocol [9]. This enables full-duplex, bidirectional synchronization, keeping the browser interface and the physical device states perfectly aligned in real time without the latency penalty of traditional HTTP request-response polling.

The evolution of the system's communication state can be mathematically formalized as a discrete-time function:

$$s_{k+1} = g(s_k, c_k), \tag{2}$$

where s_k represents the exact device state at time step k, and c_k is the most recent control command vector acquired either from acoustic inference, gesture recognition, or explicit web input.

E. Power Management and Actuation

The proposed system is powered by a 12V DC primary source, which is strategically divided into two functional branches to safely accommodate both high-power actuation and low-voltage logic. The first branch directly supplies the high-power 12V–10W LED load through an isolated relay contact path. Since this main lighting load draws approximately 0.83 A, which significantly exceeds the direct drive capability of the ESP32 GPIO pins, it is actuated via a dedicated transistor stage. This stage comprises an NPN transistor, a base current-limiting resistor, and a flyback diode connected across the relay coil to securely suppress inductive kickback. Concurrently, the second power branch is stepped down to 5V by a stable off-the-shelf buck converter module to provide sufficient power for the relay coil, the LED strip, and the ESP32 input stage. Finally, the onboard linear regulator circuitry of the ESP32 provides the 3.3V supply required by the microphone and gesture sensor. Ultimately, this hierarchical 12V/5V/3.3V power architecture ensures that the microcontroller can safely switch a practical lighting device while maintaining strict electrical isolation, thereby protecting the sensitive logic circuitry and preserving the continuous stability of the sensing and communication subsystems.

III. Multimodal Processing and Control Logic

To deploy a functional acoustic classifier on a resource-constrained MCU, the signal processing pipeline must be rigorously parameterized. The workflow seamlessly transitions from raw acoustic wave sampling to feature extraction, and finally to neural inference.

A. Acoustic Feature Extraction and Preprocessing

The initial stage of the pipeline involves digitizing and framing the acoustic environment. The microphone signal is continuously sampled at a frequency of $f_s = 16\,\text{kHz}$. Let $x[n]$ denote the discrete-time audio sequence. A pre-emphasis filter is initially applied to balance the frequency spectrum by amplifying high-frequency formants. The filter is mathematically defined as:

$$x_{pre}[n] = x[n] - \alpha x[n-1], \tag{3}$$

where the pre-emphasis coefficient is explicitly configured as $\alpha = 0.98$. A sliding window mechanism is then employed to segment the continuous audio stream into overlapping frames. The number of samples per window is defined by the frame length $T_f = 25\,\text{ms}$:

$$N = f_s T_f = 16000 \times 0.025 = 400 \text{ samples}. \tag{4}$$

Assuming a window stride (hop size) of $T_s = 20\,\text{ms}$, the hop size in samples evaluates to $H = 320$. Given an inference macro-window of 1000 ms, the total number of frames M is computed by:

$$M = \left\lfloor \frac{16000 - 400}{320} \right\rfloor + 1 = 49 \text{ frames}. \tag{5}$$

Once the audio stream is segmented, the next step is to extract robust acoustic representations using Mel-frequency cepstral coefficients (MFCCs) [8]. After applying an FFT of length 512 to each frame, the magnitude spectrum is projected onto a Mel filterbank with 32 filters. The relationship between the linear frequency f and the Mel frequency m is modeled as:

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right). \tag{6}$$

A band-pass configuration (80Hz to 4000Hz) is applied to emphasize the useful human speech band. The coefficients are subsequently computed via the discrete cosine transform (DCT). By retaining exactly 13 coefficients per frame across the 49 temporal frames, the final acoustic feature tensor maps to a structured matrix of:

$$49 \text{ frames} \times 13 \text{ MFCCs} = 637 \text{ features}. \tag{7}$$

B. 1D-CNN Architecture and Regularization

The raw output from MFCC feature extraction is a flattened 1-dimensional array of 637 elements. To exploit the temporal dependencies inherent in human speech using convolutional operations, a reshape layer must first transform this flat vector into a 2-dimensional spatial-temporal sequence. Let $X_{\text{flat}} \in \mathbb{R}^{1\times 637}$ represent the input vector. The reshape mapping is defined as:

$$X_{\text{seq}} = \text{Reshape}(X_{\text{flat}}, T, D), \tag{8}$$

where $X_{\text{seq}} \in \mathbb{R}^{T\times D}$ is the reconstructed 2D tensor, $T = 49$ denotes the temporal frames, and $D = 13$ corresponds to the frequency channels per frame.

This reshaped sequence, X_{seq}, is then processed by a 1D convolutional neural network (1D-CNN). The first 1D convolutional layer employs 8 filters with a kernel size of 3, serving as an initial spatial-temporal feature extractor, followed by a deeper layer with 16 filters. Given an input feature map $x_i[n]$, the 1D convolution operation is calculated as:

$$y_j[n] = \sigma\left(\sum_i \sum_{k=0}^{K-1} w_{j,i}[k]\, x_i[n-k] + b_j\right), \tag{9}$$

where $w_{j,i}[k]$ represents the trainable kernel weights, b_j denotes the layer bias, and $\sigma(\cdot)$ acts as the non-linear rectified linear unit (ReLU) activation function, mathematically defined as:

$$\sigma(z) = \max(0, z). \tag{10}$$

The exclusive use of ReLU prevents floating-point overflow and significantly accelerates real-time execution compared to exponential-based activation functions.

To mitigate the risk of overfitting during the training of these convolutional filters, dropout layers are explicitly interleaved within the architecture. By randomly zeroing out a specific proportion of neuron activations, the network develops more robust acoustic pathways. Mathematically, the operation on an input neuron x_i is expressed as:

$$y_i = x_i \cdot d_i, \quad d_i \sim \text{Bernoulli}(1-p), \tag{11}$$

where $p = 0.25$ is the predefined dropout rate, and d_i is a binary mask.

Following the cascade of convolutions, pooling, and dropout regularizations, the resultant high-level spatial tensor must be mapped to the fully connected Softmax layer. A flatten operation unrolls this tensor back into a 1-dimensional continuous array. Let the output of the final convolutional stage be $H \in \mathbb{R}^{T' \times C}$, where T' is the downsampled temporal dimension and $C = 16$ is the total number of filters. The flattening operation is sequentially defined as:

$$z_k = H_{i,j} \quad \text{for } k = i \cdot C + j, \tag{12}$$

yielding the projection vector $z \in \mathbb{R}^{T' \cdot C}$ required for final categorical classification.

C. Model Optimization, Quantization, and Edge Deployment

During the offline optimization phase, the network parameters were updated using the Adam optimizer with a learning rate of $\eta = 0.005$ and a batch size of 128. However, deploying the resulting 32-bit floating-point weights directly onto the ESP32 would exceed the microcontroller's static RAM restrictions. To resolve this, a post-training quantization (PTQ) scheme is employed via the Edge Impulse EON™ Compiler.

The PTQ process maps the floating-point tensors $r \in \mathbb{R}$ to 8-bit signed integer tensors $q \in \mathbb{Z} \cap [-128, 127]$. The linear quantization relationship is defined by:

$$r = S(q - Z), \tag{13}$$

where S (scale) is the step size and Z (zero-point) is the integer offset. For each layer, these parameters are derived from the dynamic range $[\min, \max]$ observed during calibration:

$$S = \frac{\max - \min}{255}, \quad Z = \text{round}\left(-128 - \frac{\min}{S}\right). \tag{14}$$

The subsequent quantization operation for any input r utilizes a clamping function to prevent numerical overflow strictly, i.e.,

$$q = \text{clamp}\left(\left\lfloor \frac{r}{S} \right\rceil + Z, -128, 127\right). \tag{15}$$

On the physical ESP32 hardware, the multiply-accumulate (MAC) operations are parallelized utilizing optimized CMSIS-NN kernels with 32-bit intermediate accumulators. The convolutional output q_{acc} is then rapidly re-quantized back to *int8* prior to the ReLU activation:

$$q_{out} = \text{clamp}\left(\text{round}(M \cdot q_{acc}) + Z_{out}, -128, 127\right), \tag{16}$$

where the fixed-point multiplier M is implemented as a bit-shift operation, effectively avoiding costly floating-point divisions on the embedded target.

Ultimately, the output from the final dense layer is normalized into a probability distribution via the Softmax function:

$$p_c = \frac{e^{z_c}}{\sum_{r=1}^{L} e^{z_r}}. \tag{17}$$

During continuous real-time execution, to suppress false positive activations induced by ambient background noise, a strict logical decision threshold $\tau = 0.70$ is enforced upon this distribution:

$$\hat{y} = \begin{cases} \arg\max_c p_c, & \text{if } \max_c p_c > \tau \\ \text{noise / ignore}, & \text{otherwise.} \end{cases} \tag{18}$$

This formulation explicitly prioritizes operational precision, ensuring that the lighting relay toggles only when acoustic confidence is exceptionally high.

D. Gesture Acquisition and Action Mapping

The PAJ7620U2 optical sensor provides discrete hexadecimal gesture codes over the I2C bus. Rather than using continuous CPU polling, which unnecessarily consumes clock cycles, the system employs hardware-interrupt-driven acquisition. An active-low signal on the INT pin instantly triggers the ESP32 to fetch the data payload. Each recognized gesture vector (e.g., swipe up, rotate, wave) is deterministically mapped to a lighting control action, encompassing on/off switching, RGB color transitioning, brightness adjustment, or animated mode activation.

The overall physical control latency from input to actuation is approximated by:

$$t_{\text{total}} = t_{\text{acq}} + t_{\text{feat}} + t_{\text{infer}} + t_{\text{act}}. \tag{19}$$

where t_{acq} is the sensor acquisition time, t_{feat} is the MFCC feature extraction delay, t_{infer} is the neural network inference time, and t_{act} represents the mechanical actuation delay of the relay.

IV. Experimental Results

A. Dataset, Classes, and Spectrogram Analysis

The custom acoustic dataset comprises six targeted labels: *bat_led* (turn on), *tat_led* (turn off), *den_do* (red), *den_vang* (yellow), *den_xanh* (green), and an ambient *noise* class. The dataset was rigorously split into 80% training and 20% test subsets to empirically validate generalization performance.

To provide a visual intuition of the acoustic features ingested by the model, Fig. 2 depicts the Mel-spectrogram representations of the five primary activation commands. These time-frequency plots illustrate how raw audio waveforms are compressed into structured 2D matrices before CNN processing.

As evident in the spectrograms, commands containing plosive and stop consonants, such as *bat_led* and *tat_led* (Fig. 2a and 2b), exhibit distinct, sharp broadband energy bursts across the frequency spectrum. Conversely, the color-specific commands (*den_do*, *den_vang*, *den_xanh*) display a significantly high degree of morphological overlap in their initial temporal frames. This overlap is physically attributed to the shared phonetic prefix "đèn". The distinguishing features for these color commands heavily rely on the subtle tonal and vowel formant transitions occurring in the latter half of the sample window. This specific temporal-spectral distribution provides theoretical validation of the architectural choice of a 1D-CNN, which is exceptionally adept at capturing localized temporal correlations within a shared feature space.

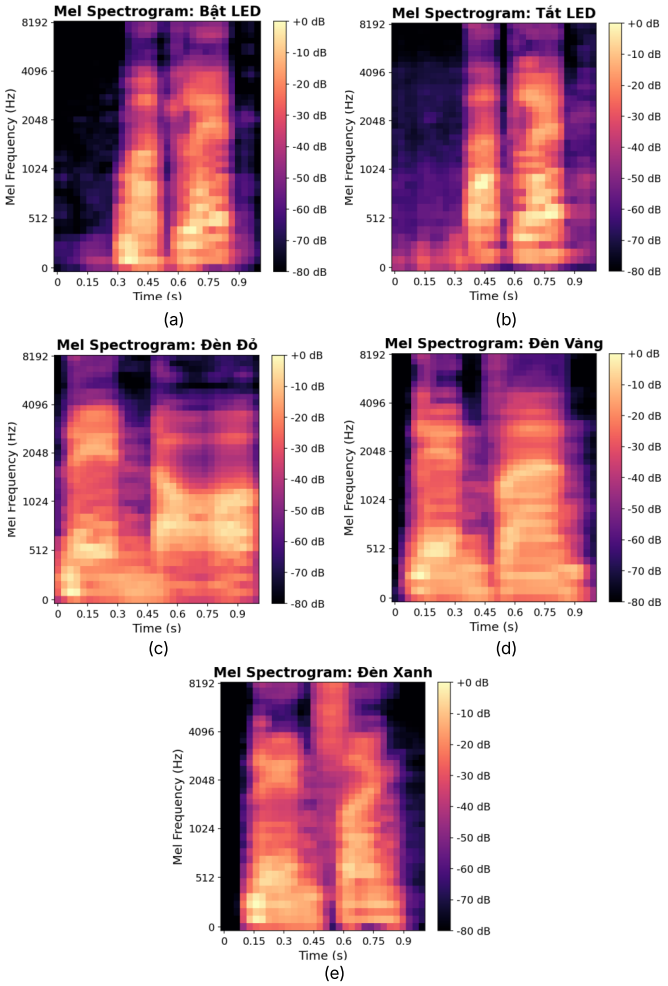


Fig. 2. Mel-spectrogram representations of the five primary voice commands, illustrating the time-frequency energy distribution prior to 1D-CNN processing. The subfigures correspond to the targeted acoustic classes: (a) *bat_led*, (b) *tat_led*, (c) *den_do*, (d) *den_vang*, and (e) *den_xanh*. These visualizations highlight the distinct broadband plosive signatures in actuation commands versus the heavily overlapping low-frequency phonetic prefixes in color-selection commands.

B. Evaluation Metrics and Practical Significance

Evaluating a KWS model embedded in a smart home environment requires metrics that directly correlate with the end-user experience. Relying solely on overall accuracy is statistically insufficient, especially when dealing with imbalanced ambient noise. Therefore, we evaluate the system using a comprehensive set of performance indicators defined by true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

First, the overall accuracy provides a baseline for the model's general classification performance across all input conditions. To address the system's robustness against unintended activations, we calculate its precision:

$$\text{Precision} = \frac{TP}{TP+FP}. \tag{20}$$

In the practical context of our contactless lighting system, precision acts as an indicator of resistance to false positives. A high precision score ensures that the lights do not randomly trigger due to background noise, overlapping conversations, or television sounds (ghost activations), which is crucial for maintaining user comfort, particularly at night.

Conversely, to evaluate the system's responsiveness when explicitly interacted with, we compute the recall:

$$\text{Recall} = \frac{TP}{TP+FN}. \tag{21}$$

Recall measures the model's immunity to false negatives. From a user perspective, a high recall score ensures that when a valid command is issued, the system executes it immediately, effectively preventing the frustration of having to repeat commands.

Recognizing the inherent mathematical trade-off between maximizing precision and recall, the F1-Score is utilized:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{22}$$

Serving as the harmonic mean of precision and recall, the F1-Score provides a single, unified metric to benchmark the model's balanced reliability. Furthermore, the area under the receiver operating characteristic curve (AUC-ROC) is used to measure the model's ability to distinguish between classes independent of a fixed decision threshold.

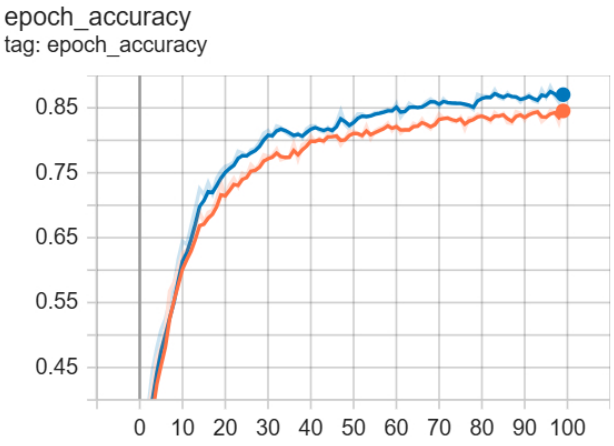
C. Model Training Dynamics

To rigorously evaluate the learning behavior of the proposed 1D-CNN, the progression of accuracy and categorical cross-entropy loss over the training epochs was recorded. Fig. 3 illustrates these optimization trajectories.

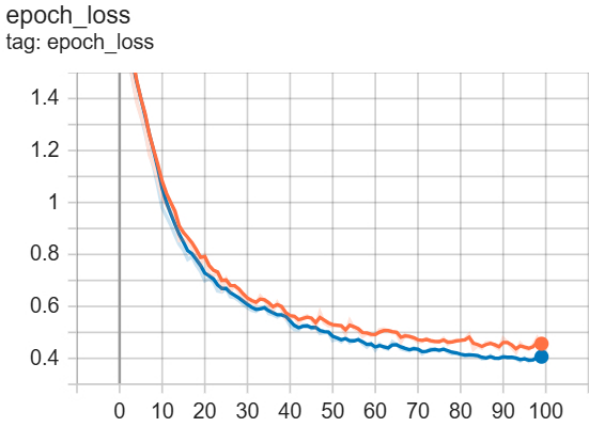
As depicted in Fig. 3(b), the training and validation losses demonstrate a steep, monotonic descent during the initial epochs, indicating a rapid acquisition of essential acoustic features. Subsequently, the loss curves asymptotically converge to a stable minimum, with minimal fluctuations. Correspondingly, Fig. 3(a) shows a swift ascent in classification accuracy, which smoothly plateaus. Crucially, the margin between the training and validation curves remains tight throughout the entire optimization phase. This narrow generalization gap empirically validates the efficacy of the dropout regularization strategy ($p = 0.25$) integrated into the convolutional layers. It confirms that the network successfully mitigates overfitting while retaining the capacity to extract highly discriminative representations from the complex MFCC feature space.

D. Inference Performance and Feature Space Analysis

Upon completion of training, the quantized Int8 1D-CNN achieved an outstanding overall validation accuracy of 88.2%, maintaining a minimal cross-entropy loss value of 0.39. A critical highlight of the model's robustness is its exceptional AUC-ROC score of 0.99. In real-world edge applications, a near-perfect AUC indicates that the model achieves a superior ability to minimize both Type I (false positive) and Type II (false negative) errors across varying operating conditions;



(a) Model Accuracy.



(b) Model Loss.

Fig. 3. Training dynamics of the 1D-CNN model. (a) Progression of training and validation accuracy. (b) Convergence of categorical cross-entropy loss over the allocated epochs.

consequently, applying the predefined high-confidence decision threshold ($\tau = 0.70$) will effectively filter out noise without suppressing legitimate voice commands. These overarching metrics, along with on-device footprint measurements, are summarized in Table I.

To visually validate the efficacy of the MFCC feature extraction and Int8 quantization pipeline, the high-dimensional acoustic features were projected onto a 2D plane utilizing dimensionality reduction (as shown in the Data Explorer module of Edge Impulse, depicted in Fig. 4).

The latent space visualization provides profound insights into the model's behavior. This high degree of linear separability in the pre-processed feature space provides empirical justification for employing a lightweight, low-complexity 1D-CNN model. Because the feature clusters are already well-defined, the subsequent neural network requires significantly fewer parameters to establish decision boundaries, thereby enabling efficient Int8 quantization without substantial degradation in

TABLE I
PERFORMANCE EVALUATION OF THE QUANTIZED 1D-CNN MODEL

Metric	**Value**
Classification Metrics (Validation Set)	
Overall Accuracy	88.2%
Area under ROC Curve (AUC)	0.99
Weighted Average Precision	0.89
Weighted Average Recall	0.88
Weighted Average F1-Score	0.88
Categorical Loss	0.39
On-Device Performance (EON™ Compiler)	
Inferencing Time	7 ms
Peak RAM Usage	3.8 KB
Flash Usage	31.2 KB

classification accuracy. More importantly, the misclassified samples (represented by red dots) are not randomly distributed; rather, they are heavily concentrated at the peripheries of adjacent clusters.

This observation mathematically correlates with the detailed confusion matrix presented in Table II. The *den_do* (red) command exhibits the lowest F1-score (0.80) and is frequently misclassified as *den_xanh* (16.9%) or *den_vang* (11.9%). Based on the feature-space analysis, this cross-confusion is acoustically justified. These commands are phonetically concise and share the identical Vietnamese prefix "đèn", leading to significant overlapping regions in their MFCC tensors and, subsequently, in the latent space. Despite minor phonetic confusions in color selection, the F1-scores for primary actuation commands (*bat_led* = 0.93, *tat_led* = 0.89) remain exceptionally high.

TABLE II
CONFUSION MATRIX AND F1-SCORES OF THE KEYWORD SPOTTING MODEL (%)

Expected \ Predicted	bat_led	den_do	den_vang	den_xanh	noise	tat_led
bat_led	91.4	0	0	0	4.9	3.7
den_do	0	69.5	11.9	16.9	0	1.7
den_vang	0	4.1	82.2	9.6	1.4	2.7
den_xanh	0	0	1.4	98.6	0	0
noise	1.2	0	0	3.5	93.0	2.3
tat_led	4.8	0	0	0	4.8	90.5
F1 SCORE	0.93	0.80	0.85	0.87	0.92	0.89

Beyond classification metrics, on-device benchmarking conclusively proves the viability of Edge AI on the ESP32 architecture. Leveraging the EON Compiler, the model achieved an ultra-low inference latency of only 7 ms per forward pass. The peak RAM footprint for tensor allocation was limited to 3.8 KB, while persistent Flash memory usage was optimized to 31.2 KB. Achieving an AUC of 0.99 with such a microscopic computational footprint represents a significant milestone in deploying reliable, untethered smart home interfaces.

E. System Memory Stability

To validate the architectural soundness of our embedded firmware, dynamic memory (heap) stability was rigorously



Fig. 4. Latent feature space visualization (Data Explorer) demonstrating the clustering of acoustic classes and decision boundaries.

profiled under two distinct operating conditions:

1) Boot-up state: Captured immediately following the initialization of the Wi-Fi stack, web server, Socket.IO channels, and sensor peripherals.
2) Full-load state: Monitored during continuous concurrent execution of I2S sampling, MFCC extraction, neural inference, and interrupt handling.

Profiling revealed that the total free heap remained abundantly available at approximately 208.65 KB post-boot, and settled at a stable 177.95 KB under sustained full load. Crucially, the maximum allocatable contiguous block size never dropped below 107.99 KB. This value indicates that the system maintains a robust memory safety margin and that heap fragmentation is exceptionally well controlled, preventing buffer overflows during long-term continuous edge operation.

F. Comparative Analysis

To rigorously evaluate the efficacy of the proposed system, we performed a comparative analysis against the state-of-the-art speech recognition framework deployed in an ESP32 by González *et al.* [4]. This comparison, summarized in Table III, focuses on architectural complexity, resource utilization, and temporal performance.

TABLE III
PERFORMANCE AND ARCHITECTURAL COMPARISON BETWEEN THE PROPOSED MULTIMODAL SYSTEM AND STATE-OF-THE-ART ESP32 IMPLEMENTATION

Metric	González *et al.* [4]	Proposed System
Interaction Modality	Unimodal	Multimodal
Network Architecture	Deep	Streamlined
Filter Density	$16 \rightarrow 32 \rightarrow 64 \rightarrow 64$	$8 \rightarrow 16$
Execution Engine	TFLite Micro	EON™ Compiler
Classification Accuracy	87.14%	88.20%
Inference Latency	28 ms (266 ms total cycle)	7 ms (257 ms total cycle)
Peak RAM Allocation	∼192 KB	∼134 KB
IoT Synchronization	Serial Monitor	WebSocket

Note: The proposed architecture achieves 3.38% reduction in total processing latency and an approximate 30% reduction in RAM footprint by substituting traditional interpreter-based execution with hardware-aware static memory allocation, while simultaneously introducing optical gesture & web synchronization via Wi-Fi.

1) Topological Optimization and Computational Efficiency

A fundamental divergence in the design philosophy lies in the chosen neural network topologies. The reference architecture uses a deep four-block Conv1D structure, with increasing filter densities (16 to 64 filters per layer). While deep hierarchical feature extraction is conventionally employed to maximize phonetic robustness across highly generalized acoustic datasets, our empirical evaluations demonstrate that for a localized, domain-specific vocabulary (e.g., Vietnamese smart lighting commands), a streamlined two-layer Conv1D architecture is optimal. By strategically capping the filter density at 16, we exponentially reduce the total number of trainable parameters and multiply-accumulate (MAC) operations required per forward pass. This topological parsimony directly yields a fourfold acceleration in core inference latency—decreasing from 28 ms in the benchmark model to an ultra-low 7 ms in our proposed system—without sacrificing classification accuracy (achieving 88.20% versus the benchmark's 87.14%).

2) Execution Engine and Deterministic Memory Allocation

The stark contrast in memory utilization highlights a critical advancement in our deployment strategy. González *et al.* [4]. report a systemic RAM allocation of approximately 192 KB. In contrast, our system operates with a significantly leaner peak footprint of $\sim$134 KB (a $\sim$30% reduction), while the AI model itself consumes a microscopic 3.8 KB (fifty times less than the benchmark model) of static RAM. The execution engine paradigm primarily drives this superior memory efficiency. The benchmark model relies on the TensorFlow Lite for Microcontrollers (TFLM) interpreter, which inherently incurs memory bloat due to dynamic tensor arenas and runtime graph management. By replacing TFLM with the hardware-aware Edge Impulse EON™ Compiler, our implementation generates source-level C++ code that employs deterministic static memory allocation. Furthermore, this memory parsimony accommodates the integration of multimodal interrupt handling. We successfully offloaded I2C and I2S data acquisition to hardware-level interrupts and direct memory access (DMA), ensuring that the ESP32's dual-core processor remains dedicated to the acoustic pipeline without inducing real-time operating system (RTOS) heap fragmentation.

3) Temporal Performance and Multimodal Utility

Beyond core classification metrics, the proposed system demonstrates a holistic improvement in the total operational cycle, reducing the end-to-end processing latency from 266 ms to 257 ms. More importantly, compressing the neural inference phase generates substantial computational headroom. This reclaimed processing bandwidth is strategically utilized to manage high-cadence optical gesture polling and full-duplex WebSocket synchronization concurrently. Although the baseline unimodal architecture relies on basic serial monitoring due to computational saturation, our optimized framework successfully delivers a production-ready multimodal IoT interface. This feature demonstrates that architectural streamlining, coupled with hardware-aware compilation, is highly effective at transforming resource-constrained microcontrollers into robust, real-time smart home hubs without performance degradation.

V. Conclusion

This paper has comprehensively presented the design and implementation of a highly optimized multimodal smart lighting system that seamlessly synergizes acoustic keyword spotting, optical gesture recognition, and WebSocket-driven telemetry on a single ESP32 edge node. By deploying a two-layer streamlined Int8-quantized 1D-CNN through the Edge Impulse EON™ Compiler, the proposed architecture effectively resolves the computational and memory bottlenecks prevalent in contemporary TinyML deployments based on interpreters.

A rigorous comparative analysis against state-of-the-art unimodal implementations demonstrates the technical superiority of our approach. Using hardware-aware static memory allocation and topological parsimony, the system achieves a competitive classification accuracy of 88.20% and a near-perfect AUC of 0.99. At the same time, the AI model itself consumes a microscopic 3.8 KB of static RAM. Most notably, the architecture delivers an ultra-low inference latency of just 7 ms. These statistics represent a fourfold acceleration in core processing speed and an approximate 30% reduction in the system memory footprint compared to existing deep-network TensorFlow Lite for Microcontrollers (TFLM) baselines.

Crucially, the extraction of this computational headroom proves that Edge AI does not need to monopolize device resources. The reclaimed processing bandwidth successfully enables the ESP32 to concurrently process hardware-interrupt-driven I2C gesture data and maintain full-duplex IoT web synchronization without inducing RTOS heap fragmentation or preemptive scheduling conflicts.

Ultimately, these outcomes definitively assert that highly optimized, compiled Edge AI can transcend basic classification tasks, serving as a reliable foundation for complex, real-time human-machine interaction applications on severely constrained microcontrollers. Future research will aim to expand the acoustic dataset to encompass highly reverberant environments, explore sensor fusion with low-power computer vision modules to broaden spatial tracking, and federate multiple edge nodes utilizing lightweight message queueing telemetry transport (MQTT) infrastructure for large-scale enterprise deployments.

References

[1] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, "A comprehensive survey on tinyml," *IEEE Access*, vol. 11, pp. 96 892–96 922, 2023.

[2] S. Chittepu, S. Martha, and D. Banik, "Empowering voice assistants with tinyml for user-centric innovations and real-world applications," *Scientific Reports*, vol. 15, no. 1, p. 15411, 2025.

[3] H. Y. Tsoo, T. M. Ting, and L. S. Chow, "Deployment of speaker identification system onto esp32-devkitc microcontroller using neural network and tensorflow lite," *SN Computer Science*, vol. 6, no. 736, 2025.

[4] M. González, S. Gutiérrez, R. Espinosa, and H. Ponce, "Deploying real-time speech recognition on esp32 using tinyml and edge impulse," *Preprint*, 2025.

[5] M. Kazeneza, A. S. Bosman, D. K. Amenyedzi, D. Hanyurwimfura, E. Ndashimye, and A. Vodacek, "Balancing complexity and performance of machine learning models for avian pests sound detection in agricultural environments," *IEEE Access*, 2025.

[6] T. N. Sainath and C. Parada, “Convolutional neural networks for small-footprint keyword spotting,” in *Proceedings of Interspeech*, 2015, pp. 1478–1482.
[7] *PAJ7620U2 Integrated Gesture Recognition Sensor Datasheet*, PixArt Imaging Inc., 2022.
[8] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.
[9] I. Fette and A. Melnikov, “The websocket protocol,” IETF, RFC 6455, 2011.
[10] *INMP441 Omnidirectional Microphone with I2S Digital Output Datasheet*, InvenSense, 2015.
[11] *WS2812B Intelligent Control LED Integrated Light Source Datasheet*, Worldsemi, 2024.
[12] *LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator Datasheet*, Texas Instruments, 2023.